Bacharelado em Ciência da Computação
Fase 3

A AURORA ESTABELECE OS PRIMEIROS SISTEMAS DA COLÔNIA

Sistema integrado de energia da colônia

Repositório no GitHub:
github.com/FIAP-CC2026/FIAP-Ciencia-da-computacao-fase-3

TRABALHO INTEGRADO

Grupo 3

Gabriel Ferreira da Silva
José Kauan Medeiros Machado
Murilo Ribeiro Falconeri
Gustavo Rocha Caxias

2026

1. Introdução

O sistema da colônia Aurora Siger reúne, em um único programa em Python, quatro tarefas: organizar os dados de energia da colônia, tomar decisões automáticas, prever a geração eólica com regressão linear e analisar se a energia gerada é suficiente para o consumo. O código usa apenas recursos básicos da linguagem, sem bibliotecas externas.

O programa segue o fluxo entrada, processamento e saída. Cada parte fica em um arquivo da pasta colonia: dados.py, regras.py, previsao.py e energia.py. O main.py junta tudo em um menu com cinco cenários prontos, uma opção para digitar os valores e outra para prever a energia eólica a partir do vento.

2. Como os dados foram organizados

Os dados ficam no arquivo dados.py, em um dicionário aninhado chamado colonia. Organizamos ele de forma hierárquica: no primeiro nível ficam as partes da colônia (energia, consumo, clima, equipamentos e historico) e, dentro de cada uma, os valores de cada subsistema:

```
colonia
├── energia       solar, eolico, reserva
├── consumo       suporte_vida, habitacao, laboratorio, comunicacao
├── clima         vento, tempestade
├── equipamentos  bateria → carga
└── historico     vento [8, 10, 12], energia_eolica [20, 25, 30]
```

A ideia foi deixar os dados parecidos com a própria colônia. A energia tem as fontes solar e eólica e a reserva, o consumo é dividido por sistema e os equipamentos guardam o estado de cada um, como a carga da bateria. Assim os outros arquivos sabem onde procurar cada informação.

Escolhemos o dicionário porque ele busca o valor direto pela chave. No Python, o dicionário funciona como uma tabela hash: a chave é transformada em um número (o hash) que indica onde o valor está guardado, então não é preciso percorrer nada. Isso dá acesso O(1) em média. Em uma lista, teríamos que olhar item por item até achar o que queremos, o que é O(n). Por exemplo, a geração solar sai de colonia["energia"]["solar"]. A função obter(colonia, *caminho) faz isso com várias chaves seguidas: obter(colonia, "energia", "solar") entra em energia e depois em solar.

No histórico usamos listas porque a ordem importa. Cada posição é uma medição, e a posição 0 do vento corresponde à posição 0 da energia_eolica. A regressão da seção 4 usa as duas listas juntas, em pares.

Com os dados atuais, total_geracao soma solar e eólico e dá 65 kW (40 + 25). A reserva fica de fora porque ela guarda energia, mas não gera. O total_consumo soma os quatro sistemas e dá 70 kW, e o listar_alertas devolve ["bateria"], porque a carga da bateria está em 15% e o limite é 20%. Ou seja, já dá para ver que a colônia consome mais do que gera. Esses valores aparecem no início do programa, e o histórico é usado na previsão da seção 4.

3. Quais regras de decisão foram utilizadas

As regras ficam no arquivo regras.py, na função decidir(energia, consumo, previsao). Primeiro o código calcula três condições simples e depois combina essas condições com if, elif e else:

- energia_baixa: geração abaixo de 50 kW;
- consumo_alto: consumo acima de 60 kW;
- previsao_ruim: previsão eólica desfavorável.

Condição	Ação
geração < 20 kW	MODO ECONOMIA CRÍTICO: manter apenas sistemas essenciais
energia_baixa E consumo_alto E previsao_ruim	MODO ECONOMIA: desligar sistemas não essenciais
energia_baixa OU consumo maior que a geração	ALERTA: reduzir consumo
nenhuma das anteriores	NORMAL: manter sistemas ligados

Tabela 1. Regras de decisão.
Fonte: elaborado pelo grupo (2026).

A situação mais grave é testada primeiro. Com geração de 40 kW e consumo de 70 kW, que é o exemplo do enunciado, a resposta é "ALERTA: reduzir consumo". Mesmo quando a geração não está baixa, se o consumo passar dela o sistema também pede para reduzir o consumo.

Quando a decisão é de economia, a função sistemas_a_desligar define o que desligar. A ordem de prioridade é suporte à vida, habitação, comunicação e laboratório. Suporte à vida e habitação são essenciais e nunca são desligados. Os outros são desligados do menos para o mais importante: primeiro o laboratório, depois a comunicação. O fluxograma da página seguinte mostra o caminho completo da decisão.

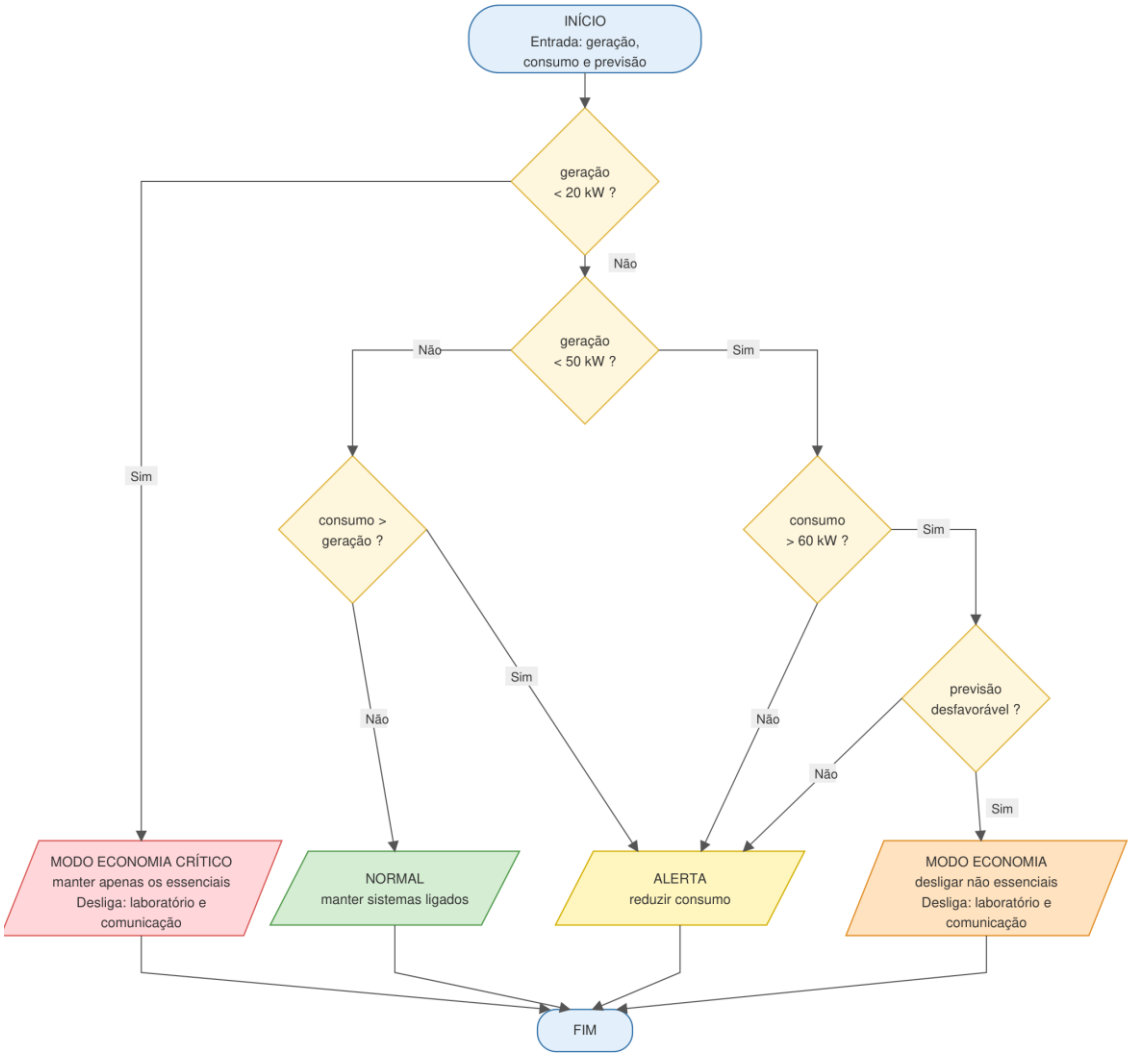


Figura 1. Fluxograma das regras de decisão.
Fonte: elaborado pelo grupo (2026).

4. Qual foi o modelo de previsão aplicado

Usamos regressão linear simples para estimar a energia eólica a partir do vento, no arquivo previsao.py. A reta tem a forma energia = a × vento + b, e os coeficientes são calculados pelo método dos mínimos quadrados:

```
a = soma((vento - média_vento) × (energia - média_energia))
    / soma((vento - média_vento)²)
b = média_energia - a × média_vento
```

Conta passo a passo com o histórico vento = [8, 10, 12] e energia = [20, 25, 30]:

- médias: vento 10 e energia 25;
- diferenças em relação à média: vento -2, 0 e 2; energia -5, 0 e 5;
- numerador: (-2)(-5) + 0 × 0 + 2 × 5 = 20;
- denominador: 4 + 0 + 4 = 8;
- a = 20 / 8 = 2,5 e b = 25 - 2,5 × 10 = 0.

A reta fica energia = 2,5 × vento. Para vento de 11 m/s, a previsão é 2,5 × 11 = 27,5 kW.

A função classificar_previsao transforma esse número em "favoravel", quando a previsão é de pelo menos 25 kW, ou "desfavoravel", quando fica abaixo disso. É esse valor que entra nas regras de decisão. O ajuste exige pelo menos duas observações com valores de vento diferentes, e o programa avisa quando o vento pedido está fora da faixa do histórico (8 a 12 m/s), onde a estimativa é menos confiável.

5. Como o sistema ajuda a melhorar o uso de energia

A função analisar_energia, no arquivo energia.py, compara geração e consumo:

- consumo maior que geração: "ALERTA: consumo maior que geração";
- geração maior que consumo: "SUGESTÃO: armazenar energia excedente";
- valores iguais: "OK: geração e consumo equilibrados".

O programa também mostra o saldo (geração menos consumo) e quanto cada fonte representa na geração total, com a função participacao_fontes. Juntando isso com a previsão e as regras, o sistema deixa de apenas reagir ao que já aconteceu: quando o vento previsto é fraco, ele entra em modo economia antes de a energia acabar.

Cenário	Geração / consumo	Previsão eólica	Análise	Decisão
1. Pouca energia e consumo alto	40 / 70 kW	27,5 kW (favorável)	consumo maior que geração	ALERTA: reduzir consumo
2. Sobra de energia	80 / 30 kW	30,0 kW (favorável)	armazenar excedente	NORMAL
3. Pouca energia, consumo alto e pouco vento	40 / 70 kW	20,0 kW (desfavorável)	consumo maior que geração	MODO ECONOMIA
4. Geração e consumo iguais	60 / 60 kW	25,0 kW (favorável)	equilibrados	NORMAL
5. Tempestade de poeira	15 / 60 kW	20,0 kW (desfavorável)	consumo maior que geração	MODO ECONOMIA CRÍTICO

Tabela 2. Resultado dos cenários prontos do programa.
Fonte: elaborado pelo grupo (2026).

Os cenários 1 e 3 têm a mesma geração e o mesmo consumo, e a diferença está no vento previsto. No cenário 3, a previsão desfavorável faz o sistema desligar o laboratório e a comunicação para preservar o suporte à vida. No cenário 2, a sobra de energia é indicada para armazenamento, evitando desperdício.

6. Considerações finais

O sistema junta estruturas de dados, regras lógicas, regressão linear e análise de energia em um programa simples de usar e de testar. Os dados são fictícios, escolhidos para mostrar cada situação, e as unidades são ilustrativas. Como próximos passos, o histórico de vento poderia ter mais medições e a previsão poderia incluir também a energia solar.